

42 프로젝트 기록 요약

이 문서는 Notion 페이지 "42 프로젝트 기록들"을 간결하게 정리한 요약본입니다.

각 프로젝트에 대한 간단한 설명, 주요학습 내용과 사용한 핵심기술을 요약했습니다.

Notion 주소: <https://bit.ly/48zlOwk>

Github 주소: <https://github.com/tmdgus1dnl/42projects>

Minishell

설명: 간소화된 셸 구현

주요 내용:

- 터미널에서 작동하는 가벼운 셸(shell)을 C언어로 구현했습니다.
- 사용자가 입력하는 문자열을 파싱(tokenization)하여 명령어와 인자를 분리하고, 콤마(","), 파이프(|), 리다이렉션(<, >)등을 처리했습니다.
- 환경변수와 내장명령어(echo, cd, export 등)를 구현하여 기존 /bin/sh 와 비슷한 기능을 제공합니다.
- 각 명령 실행 시 자식 프로세스로 분기(fork)하여 처리했고, 명령 실패시 적절한 오류메시지를 출력하도록 했습니다.
- 명령어 수행완료 혹은 프로그램 종료 시 최종 메모리 해제를 통해 메모리 누수를 방지했습니다.

C++ Module

설명: C++ 핵심기능들

주요 내용:

- 객체지향언어 C++의 클래스와 인스턴스, 상속, 다형성같은 개념을 학습하며, 각 모듈에서 직접 클래스를 설계하여 OOP의 중요성을 학습했습니다.
- C 스타일 포인터와 참조(&)의 차이를 익히고, 생성자와 소멸자, 복사생성자, 대입연산자 등을 구현하여 RAII 규칙을 익혔습니다.
- 템플릿과 템플릿 특수화를 통해 자료형에 독립적인 코드를 작성하고, 표준 템플릿 라이브러리(STL)의 map, stack, deque 등의 컨테이너를 분석하였습니다.
- 컨테이너의 정렬 효율성 비교를 위해 시간복잡도 분석과 함께 Ford-Johnson 알고리즘을 구현했습니다. 이는 주어진 수를 작은 그룹으로 분할한 뒤 병합하여 정렬하는 방법으로, 최악의 경우 비교 횟수를 줄이기 위한 기법입니다.

ft_transcendence

설명: 풀스택 웹 프로젝트

주요 내용:

- TypeScript와 Node.js를 사용하여 웹 기반 탁구게임을 구현했습니다.
- 프론트는 Vite로 빌드된 싱글페이지(SPA) 구조이며, 백엔드는 Express.js 위에 작성되었습니다.
- 프론트를 맡아 프레임워크를 사용하지 않고 SPA 구조를 만들기 위해 라우터를 구현했습니다.
- 라우팅은 브라우저의 History API를 이용하여 구현했으며, 페이지 전환시 전체를 리로드 하지않고 URL을 변경할수 있도록 pushState/popState 이벤트를 관리했습니다.

Inception

설명: 도커기반 웹 환경 구축

주요 내용:

- 가상화와 컨테이너화의 차이를 이해했습니다. 전통적 가상머신은 하이퍼바이저가 전체 운영체제를

가상화하는 반면, 컨테이너는 호스트 커널을 공유하여 오버헤드가 적습니다.

- Docker를 사용해 Nginx, WordPress + PHP-FPM, MariaDB 세 개의 컨테이너를 빌드하고, docker-compose를 사용하여 서로 연결했습니다.
- 각 서비스의 Dockerfile을 작성하여 이미지 레이어의 구성과 Union 파일시스템의 개념을 배웠습니다.
- 도커 볼륨을 사용해 데이터의 영속성을 보장했고, 도커 네트워크 설정을 통해 컨테이너 간 통신을 구성했습니다.

Born2beroot

설명: 시스템관리 및 보안

주요 내용:

- 리눅스 서버를 설치하고 기본적인 시스템관리 및 보안정책을 수립했습니다.
- sudo 권한설정, UFW(방화벽)를 이용한 포트관리, SSH 접근제한과 암호정책 설정을 실습했습니다.
- APT vs aptitude의 차이를 실습하고, 파티션을 LVM으로 구성하여 디스크공간을 논리적으로 조정했습니다.
- VirtualBox를 사용하여 가상머신을 구축하고, 포트포워딩과 네트워크 설정을 통해 외부 접속을 테스트했습니다.
- Fail2ban 서비스를 구현하여 반복적인 악성 외부 접속 시도시 로그인 시도를 차단했습니다.

Philosophers

설명: 동시성 문제해결

주요 내용:

- 고전적인 철학자 식사문제(Dining Philosophers Problem)를 통해 동시성에 대해 학습했습니다.
- 첫번째 버전에서는 POSIX 스레드와 뮤텁스를 사용하여 철학자(스레드)가 포크를 잡고 내려놓는 동작을 조율했습니다.
- 두번째 버전에서는 프로세스와 세마포어를 사용하여 구현했습니다.
- 두 버전 모두 공통자원(포크) 접근을 제한하여 Data race를 방지하고, 교착 상태(deadlock)를 피하는 대칭 깨기 알고리즘을 적용했습니다.
- 시간 측정과 스케줄러에 대한 이해, 실행 객체간 컨텍스트 스위칭에 따른 오버헤드 등을 실습을 통해 익혔습니다.

Webserv

설명: HTTP 서버 구현

주요 내용:

- C++로 HTTP/1.1 서버를 구현했습니다.
- 소켓 프로그래밍과 I/O 멀티플렉싱(MacOS 의 kqueue)을 사용하여 여러 클라이언트의 요청을 비동기로 처리했습니다.
- Nginx 스타일의 설정파일을 파싱하여 가상 호스트, 루트 디렉터리, 리스닝 포트등을 지정할 수 있도록 했습니다.
- 요청라인을 분석해 메서드(GET, POST, DELETE)와 경로, 헤더를 해석했으며, 정적파일 제공과 동적파일 제공을 위한 CGI 실행을 지원했습니다.
- 에러페이지 생성과 keep-alive 처리, chunked 인코딩, 파일 업로드등 HTTP의 세부사항을 직접 구현해 프로토콜에 대한 이해를 높였습니다.